Hri7hik H4cks — Writer's Guide

The complete blogger's handbook for Hugo + Obsidian + automation

---

1. What this is

A foolproof guide for writing, previewing, and publishing posts on the Hri7hik H4cks blog. You write in **Obsidian** (or any markdown editor), the automation converts it to Hugo format, and you get a fast static blog at the end. Zero command-line expertise required after the first-time setup.

What you'll learn

- How to set up the project once
- Your daily writing loop
- All four post templates and how to use them
- How to add code, callouts, images, badges
- How to preview and publish
- How to fix every common error

---

2. First-time setup (one-time only)

You only do this **once** per machine. Skip to Section 3 if already done.

2.1 Prerequisites

You need on your system:

Tool	Why	Install on Kali/Debian
Hugo (extended)	Static site generator	`sudo apt install hugo`
Python 3	Conversion engine	Pre-installed on Kali
Git	Theme submodule	Pre-installed
inotify-tools	Live file watching (optional but recommended)	`sudo apt install inotify-tools`

Verify:

```
hugo version       # should say "extended"
python3 --version  # 3.10+
git --version
```

2.2 Clone the repo (if you haven't)

```
git clone <your-repo-url> ~/Hri7hik_H4cks
cd ~/Hri7hik_H4cks
git submodule update --init --recursive   # CRITICAL — pulls PaperMod theme
```

> **Skipping the `submodule` step gives you "page not found" errors and missing layouts. If you see those, run that command.**

2.3 Create the Python virtual environment

Kali blocks system-wide pip. You **must** use a venv:

```
cd ~/Hri7hik_H4cks
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

You'll know the venv is active when your prompt shows `(venv)` at the start.

> **Every new terminal needs `source venv/bin/activate` before running workflow commands.** Forgetting this is the #1 cause of confusing errors.

2.4 Initial structure check

```
./scripts/workflow.sh setup
```

Creates `obsidian-vault/posts/`, `obsidian-vault/attachments/`, `static/images/`, `content/posts/` if missing.

2.5 Sanity test

```
./scripts/workflow.sh check
./scripts/workflow.sh serve
```

Open `http://localhost:1313/` — you should see the blog. Press `Ctrl+C` to stop.

If anything fails here, jump to **Section 14: Troubleshooting**.

---

3. The daily writing loop

Your day-to-day workflow is **three commands**:

```
source venv/bin/activate                              # activate env
./scripts/workflow.sh new "My Post Title" tutorial    # create post
./scripts/workflow.sh serve                           # live preview
```

That's it. The serve command stays running while you write. Save in Obsidian → browser refreshes automatically.

When done writing:

```
./scripts/workflow.sh publish my-post-title           # mark as live
./scripts/workflow.sh build                           # production build
```

---

4. Creating posts: the `new` command

```
./scripts/workflow.sh new "Post Title" [template-kind]
```

What it does

1. Slugifies your title — `"SQL Injection Basics"` → `sql-injection-basics`
2. Creates `obsidian-vault/posts/sql-injection-basics.md`
3. Prepends Hugo front matter (title, date, draft: true, empty tags/categories)
4. Appends the chosen template body (with placeholders ready to fill in)
5. Tells you to edit it in Obsidian

Template kinds

Kind	When to use
`ctf-walkthrough` (default)	Hack The Box, TryHackMe, picoCTF write-ups
`tutorial`	How-to guides, step-by-step lessons
`security-analysis`	CVE deep-dives, malware analysis, research
`quick-reference`	Cheat sheets, command references

Examples

```
./scripts/workflow.sh new "Lame HTB Walkthrough" ctf-walkthrough
./scripts/workflow.sh new "Burp Suite Intercept Tutorial" tutorial
./scripts/workflow.sh new "Log4Shell Analysis" security-analysis
./scripts/workflow.sh new "Nmap Cheat Sheet" quick-reference
```

If you omit the kind, you get `ctf-walkthrough` by default.

If the slug already exists, the command refuses to overwrite. Pick a different title or delete the old file manually.

---

5. Templates explained

Each template lives in `obsidian-templates/` and is a regular markdown file with **placeholders** (text in `{{...}}`) and pre-built sections.

How placeholders work

Placeholders look like `{{Machine Name}}` or `{{Difficulty Level}}`. They are **plain text** — Obsidian doesn't process them. After you scaffold a post, find each `{{...}}` and replace it with your actual content. Use Ctrl+F in Obsidian to find them all.

5.1 `ctf-walkthrough` — for CTF write-ups

Pre-built sections:

- **Information callout** (Platform, Difficulty, Target IP, Objective)
- **Table of Contents** (links to each section below)
- **Introduction**
- **Reconnaissance** (port scan, service enumeration)
- **Initial Access** (exploitation)
- **Privilege Escalation**
- **Flag Finding**
- **Summary**

Replace placeholders like `{{CTF Name}}`, `{{Machine Name}}`, `{{Platform Name}}`, `{{Difficulty Level}}`, `{{Target IP}}` near the top.

5.2 `tutorial` — for how-to content

Pre-built sections:

- **Tutorial Information callout** (Category, Difficulty, Prerequisites, Time Estimate)
- Table of Contents
- Introduction / What you'll learn
- Prerequisites
- Step-by-step body
- Conclusion

5.3 `security-analysis` — for research / CVE write-ups

Pre-built sections:

- **CVE/Vulnerability Information callout** (CVE ID, CVSS, Affected systems)

- Executive summary
- Technical analysis (root cause, attack vector)
- Proof of concept
- Mitigation / recommendations

5.4 `quick-reference` — for cheat sheets

Pre-built sections:

- Tool / topic header
- Quick command tables
- Common flags / options
- Examples grouped by use case

Customizing templates

Templates are plain markdown files. To tweak one, edit it directly:

```
nano obsidian-templates/tutorial.md
```

Save. Next `new` command picks up your changes. To create a brand new template:

```
cp obsidian-templates/tutorial.md obsidian-templates/research-paper.md
# edit it
./scripts/workflow.sh new "My Research" research-paper
```

The kind name is the filename without `.md`.

---

6. Writing in Obsidian

Open `~/Hri7hik_H4cks/obsidian-vault/` as a vault in Obsidian. Your posts go in the `posts/` subfolder. Images go in `attachments/`.

6.1 Front matter (top of every file)

The block between `---` markers at the top of your markdown file. Controls metadata.

```
---
title: "Your Post Title"
date: 2026-05-04T10:00:00Z
draft: true
categories: ["CTF", "Walkthrough"]
tags: ["pentesting", "smb", "exploitation"]
difficulties: ["beginner"]
platforms: ["HackTheBox"]
tools: ["nmap", "metasploit"]
description: "One-line summary used in previews and SEO"
---
```

Field	Required	Notes
`title`	Yes	Shown as page title and in listings
`date`	Yes	ISO 8601. Auto-set by `new` command
`draft`	Yes	`true` = hidden in production. `false` = live
`categories`	No	Broad buckets: CTF, Tutorial, Analysis
`tags`	No	Specific topics; mix freely
`difficulties`	No	`beginner`, `intermediate`, `advanced`
`platforms`	No	`HackTheBox`, `TryHackMe`, `picoCTF`, `VulnHub`
`tools`	No	`nmap`, `burp suite`, `metasploit`, etc.
`description`	No	Recommended — used in SEO previews

These are the **5 taxonomies** of the blog. Tag well — they generate browse-by-tag pages automatically.

6.2 Obsidian callouts (auto-converted)

Obsidian's callout syntax converts to styled HTML boxes:

```
> [!info] Important Note
> This is an informational callout.

> [!warning] Be careful
> Pay attention to this.

> [!success] It worked!
> You did it.

> [!danger] Critical
> This will break things.
```

Becomes: - `info` → blue box with ℹ️ - `warning` → yellow/amber box with ⚠️ - `success` → green box with ✅ - `danger` → red box with 🚨

The converter handles this automatically — write Obsidian-flavor, get Hugo HTML.

6.3 Wikilinks

Obsidian's `[[Other Post]]` syntax converts to standard markdown links pointing to the slugified path. Works for both internal and external links.

6.4 Images

Drop images into `obsidian-vault/attachments/`. Reference them in markdown:

```
![Network Diagram](attachments/network-diagram.png)
```

Or paste directly into Obsidian — it auto-saves to the attachments folder.

The converter: 1. Copies the image to `static/images/` 2. Resizes to max 1200px wide 3. Re-encodes as JPEG at 85% quality (smaller file size) 4. Rewrites the link in the converted Hugo markdown

You don't have to think about this — just drop and reference.

6.5 Code blocks

Standard fenced code with language hint:

````
```bash
nmap -sC -sV 10.10.10.10
```

```python
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```
````

The blog auto-adds: - Syntax highlighting - Copy-to-clipboard button (top right of every block) - Dark theme styling

---

7. Hugo shortcodes — when you need extra power

Shortcodes are the `{{< name >}}` snippets that render special components. Use them when plain markdown isn't enough.

7.1 Code block with title and language

```
{{< code language="bash" title="Port scan" >}}
nmap -sC -sV -oA scans/nmap 10.10.10.10
{{< /code >}}
```

7.2 Terminal-style block

```
{{< terminal >}}
$ whoami
root
$ id
uid=0(root) gid=0(root)
{{< /terminal >}}
```

Renders as a green-on-black terminal block.

7.3 Tool badge

```
{{< tool "nmap" >}}    {{< tool "burp suite" >}}    {{< tool "metasploit" >}}
```

Renders as small inline badges.

7.4 Difficulty badge

```
{{< difficulty "beginner" >}}
{{< difficulty "intermediate" >}}
{{< difficulty "advanced" >}}
```

Color-coded: green / yellow / red.

7.5 Callout box (alternative to Obsidian syntax)

```
{{< callout type="info" title="Note" >}}
Body content here.
{{< /callout >}}
```

Types: `info`, `warning`, `success`, `danger`.

7.6 Image with caption

```
{{< image src="/images/diagram.png" caption="Network topology" >}}
```

---

8. Built-in HTML/CSS classes

If you write raw HTML in markdown (it's enabled), these classes are styled:

Class	Usage
`difficulty-badge difficulty-beginner`	Green badge
`difficulty-badge difficulty-intermediate`	Yellow badge
`difficulty-badge difficulty-advanced`	Red badge
`callout callout-info`	Blue info box
`callout callout-warning`	Yellow warning box
`callout callout-success`	Green success box
`callout callout-danger`	Red danger box
`terminal`	Black terminal-style block
`tool-badge`	Small tool tag

Example:

```
<div class="callout callout-warning">
  <div class="callout-title">⚠️ Important</div>
  Don't run this on production.
</div>

<span class="tool-badge">sqlmap</span>
<span class="tool-badge">hydra</span>
```

Stick to Obsidian callouts and shortcodes for 95% of cases — raw HTML is a fallback.

---

9. The auto-conversion pipeline

When you save in Obsidian, this happens behind the scenes:

```
obsidian-vault/posts/foo.md   (you edit here)
        |
        | inotify file watcher detects change
        v
scripts/obsidian_to_hugo_converter.py runs
        |
        | - parses Obsidian callouts → HTML
        | - copies images to static/images/
        | - optimizes images (resize 1200px, JPEG 85%)
        | - converts wikilinks → markdown links
        | - generates/preserves front matter
        v
content/posts/foo.md          (Hugo reads this)
        |
        v
Hugo rebuilds in-memory
        |
        v
Browser auto-reloads at localhost:1313
```

You see your changes in **under a second**.

If `inotify-tools` isn't installed, you only get Hugo's hot-reload (which watches `content/posts/` directly). To get Obsidian-source live reload, install it.

---

10. Previewing your blog

```
./scripts/workflow.sh serve
```

What this does:

1. Verifies dependencies
2. Runs incremental conversion (skips if nothing changed)
3. Starts the Obsidian file watcher in the background
4. Boots Hugo dev server with `--renderToMemory --gc --buildDrafts`
5. Opens at `http://localhost:1313/`

While it's running:

- **Save in Obsidian** → see changes within ~500ms
- **Edit** `assets/css/custom.css` → instant CSS reload
- **Edit** `hugo.toml` → Hugo restarts automatically
- **Press Ctrl+C** → stops everything cleanly (watcher + server both)

If port 1313 is busy

The script auto-detects this and falls back to 1314, 1315, etc. To force-claim 1313 (kill whatever's on it):

```
KILL_PORT=1 ./scripts/workflow.sh serve
```

To use a different port from the start:

```
HUGO_PORT=4000 ./scripts/workflow.sh serve
```

---

11. Drafts and publishing

Every new post starts as `draft: true` in its front matter. Drafts:

- Are visible during `serve` (because of `--buildDrafts`)
- Are **hidden** in production builds
- Show up in `stats` count

When ready to go live:

```
./scripts/workflow.sh publish my-post-slug
```

This:

1. Finds `obsidian-vault/posts/my-post-slug.md`
2. Changes `draft: true` → `draft: false`
3. Forces a reconvert

To list current drafts:

```
./scripts/workflow.sh publish
```

(no slug = list mode)

To go back to draft, edit the file in Obsidian and flip the flag manually.

---

12. Production build

```
./scripts/workflow.sh build
```

What you get:

- Forces full conversion (no incremental skip)
- Runs `hugo --minify --gc --buildExpired --buildFuture`
- Outputs to `public/`
- Reports build time + total size + HTML page count

The `public/` directory is what you deploy. Drag-drop to Netlify / Vercel / GitHub Pages, or rsync to your server:

```
rsync -avz --delete public/ user@host:/var/www/html/
```

> **Drafts are excluded automatically.** Want to ship a post? `publish` first, then `build`.

---

13. Other commands

`convert`

Run conversion without starting the server. Useful when you want to inspect the generated `content/posts/` files.

```
./scripts/workflow.sh convert
FORCE_CONVERT=1 ./scripts/workflow.sh convert    # skip incremental check
```

`watch`

File watcher only, no Hugo server. Useful when you're running Hugo separately or just want to verify conversions are firing.

```
./scripts/workflow.sh watch
```

`stats`

Quick health check:

```
./scripts/workflow.sh stats
```

Shows: total posts, drafts vs published count, total word count, site size on disk, image count.

`clean`

Wipes generated files. Asks for confirmation by default.

```
./scripts/workflow.sh clean              # prompts y/N
FORCE=1 ./scripts/workflow.sh clean      # no prompt
```

Removes: `content/posts/*`, `static/images/*`, `public/`, `resources/`. Your Obsidian vault is **never** touched.

`setup`

Idempotent directory setup. Safe to run any time.

`check`

Verifies dependencies. Run it if `serve` is misbehaving.

`help`

Full command reference inline.

---

14. Environment variables (advanced)

Set these in front of any command:

Variable	Default	What it does
`HUGO_PORT`	`1313`	Port for dev server
`HUGO_BIND`	`0.0.0.0`	Bind address
`KILL_PORT`	`0`	If `1`, kill anything on `HUGO_PORT` before serving
`FORCE_CONVERT`	`0`	If `1`, run converter even if up-to-date
`FORCE`	`0`	If `1`, skip `clean` confirmation prompt

Examples:

```
HUGO_PORT=8080 ./scripts/workflow.sh serve
KILL_PORT=1 HUGO_PORT=1313 ./scripts/workflow.sh serve
FORCE_CONVERT=1 ./scripts/workflow.sh build
FORCE=1 ./scripts/workflow.sh clean
```

---

15. Troubleshooting

"externally-managed-environment" / "error: externally-managed-environment"

You're on Kali (or Debian 12+) and tried to install Python packages without a venv. Fix:

```
cd ~/Hri7hik_H4cks
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Every new terminal: `source venv/bin/activate` first.

"Page not found" / Hugo warns about missing layouts

PaperMod theme submodule wasn't initialized.

```
git submodule update --init --recursive
```

Then restart `serve`.

"Unable to locate config file or config directory"

You ran `hugo` (or `workflow.sh`) from the wrong directory. The script handles this automatically now — always invoke from anywhere using:

```
~/Hri7hik_H4cks/scripts/workflow.sh serve
```

Or `cd ~/Hri7hik_H4cks` first.

"address already in use" / port 1313 busy

Either:

```
KILL_PORT=1 ./scripts/workflow.sh serve     # claim 1313 forcibly
```

Or let the script auto-pick a free port (it does by default — read the `[WARN]` line, it tells you the new port).

"inotifywait missing"

Optional but recommended:

```
sudo apt install inotify-tools
```

Without it, you lose live Obsidian-source reload but Hugo's content watcher still works.

"No Obsidian notes found"

You haven't created any posts yet. Run:

```
./scripts/workflow.sh new "My First Post"
```

Conversion ran but post doesn't appear

Two common causes:

1. `draft: true` and you're running `build` (which excludes drafts). Either `publish` it or use `serve` (drafts visible there).
2. `date:` is in the future and `--buildFuture` isn't set. The script always uses `--buildFuture` so this is rare.

Images not showing up

Make sure they live in `obsidian-vault/attachments/` and the markdown reference is `attachments/image.png` (or whatever path Obsidian inserts). The converter copies on every run — check `static/images/` after a conversion to confirm.

Template not found

```
./scripts/workflow.sh new "Title" my-template
```

…says "Template 'my-template' not found". List available kinds:

```
ls obsidian-templates/
```

The kind is the filename without `.md`.

Conversion errors / Python tracebacks

Activate the venv:

```
source venv/bin/activate
```

Reinstall packages if needed:

```
pip install --upgrade -r requirements.txt
```

---

16. Cheat sheet

Daily commands

```
source venv/bin/activate                          # always first
./scripts/workflow.sh new "Title" tutorial        # create
./scripts/workflow.sh serve                       # preview
./scripts/workflow.sh publish my-post-slug        # publish
./scripts/workflow.sh build                       # production
```

Templates

Kind	For
`ctf-walkthrough`	CTF write-ups
`tutorial`	How-to content
`security-analysis`	CVE / research
`quick-reference`	Cheat sheets

Front matter must-haves

```
---
title: "..."
date: YYYY-MM-DDTHH:MM:SSZ
draft: true | false
tags: [...]
categories: [...]
description: "..."
---
```

Obsidian callouts

```
> [!info] Title
> [!warning] Title
> [!success] Title
> [!danger] Title
```

Hugo shortcodes

```
{{< code language="bash" title="..." >}}...{{< /code >}}
{{< terminal >}}...{{< /terminal >}}
{{< tool "name" >}}
{{< difficulty "level" >}}
{{< callout type="info" title="..." >}}...{{< /callout >}}
{{< image src="..." caption="..." >}}
```

Useful URLs

URL	What
`http://localhost:1313/`	Local preview
`http://localhost:1313/posts/`	All posts
`http://localhost:1313/tags/`	Tag browser
`http://localhost:1313/categories/`	Category browser

---

17. Recommended writing habits

- **Use the `new` command, never copy templates manually** — gets the slug, date, and front matter right.
- **Tag aggressively** — taxonomies generate browse pages for free.
- **Set `description:` on every post** — it's the SEO snippet and link preview.
- **Keep drafts as drafts** until you've reread them once. `serve` shows them; `build` hides them.
- **Run `stats`** before a build — confirms count and catches accidentally-still-draft posts.

- **Resize huge screenshots before pasting** — the converter caps at 1200px but starting smaller is faster.
- **Commit often** — `git add obsidian-vault/posts/your-post.md && git commit -m "draft: foo"`. The converter output (`content/posts/`) is regenerated, but committing both is fine.

End of guide. Bookmark this PDF. When in doubt, check Section 15 (troubleshooting). When everything works, the loop is `new` → write → `serve` → `publish` → `build`. Happy hacking.